E-Clearance System: Frontend API Integration Guide

Document Version: 1.1 (Production Environment Specification)
Target Audience: Frontend Engineering Team
Scope: HTTP Endpoints, Payload Schemas, Authentication Interceptors, Client-Side Routing, and State Handling

---

1. System Architecture and Authentication

1.1 API Environments, Server Roots, and Documentation

The backend is deployed on Render and backed by a cloud MySQL database on Aiven.

Server Host vs. API Base URL

It is important to understand the distinction between the server origin and the API client base URL:

- **Server Origin (Host Root):** `https://e-clearance-system-api.onrender.com`
  - Hosts system-level routes such as the Django Admin (`/admin/`), Health checks (`/healthz`, `/health/`), and OpenAPI documentation.
- **REST API Base URL (Client Prefix):** `https://e-clearance-system-api.onrender.com/api/v1/`
  - All application REST endpoints (`/login/`, `/me/`, `/student-clearances/`, `/student-clearance-items/`, etc.) are mounted exclusively under `/api/v1/`.
  - Setting your HTTP client (Axios / Fetch) `baseURL` to this `/api/v1/` path allows you to call relative paths cleanly (e.g., `apiClient.post('/login/', ...)` instead of repeating `/api/v1/` on every call).

Interactive API Documentation

- **Swagger UI:** `https://e-clearance-system-api.onrender.com/api/docs/swagger/`
- **Redoc:** `https://e-clearance-system-api.onrender.com/api/docs/redoc/`
- **OpenAPI Raw Schema (JSON/YAML):** `https://e-clearance-system-api.onrender.com/api/schema/`

Local Development Fallback

- **Local Server Root:** `http://localhost:8000`
- **Local API Base URL:** `http://localhost:8000/api/v1/`
- **Local Swagger UI:** `http://localhost:8000/api/docs/swagger/`

Frontend Environment Variable Configuration (`.env`)

```
# Vite Configuration (.env.production / .env.local)
VITE_API_BASE_URL=https://e-clearance-system-api.onrender.com/api/v1/

# Next.js / Create-React-App Alternative
NEXT_PUBLIC_API_BASE_URL=https://e-clearance-system-api.onrender.com/api/v1/
REACT_APP_API_BASE_URL=https://e-clearance-system-api.onrender.com/api/v1/
```

1.2 Authentication and Header Management

The backend enforces token-based authorization via JSON Web Tokens (JWT).

- **Token Lifetimes:** Access Token: 15 minutes | Refresh Token: 31 days.
- Upon successful authentication, store the token securely and attach it via the standard `Authorization` header:

```
Authorization: Bearer <auth_token>
```

Axios Interceptor Configuration Example

```
import axios from "axios";

const API_BASE_URL =
  import.meta.env.VITE_API_BASE_URL ||
  "https://e-clearance-system-api.onrender.com/api/v1/";

const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    "Content-Type": "application/json",
  },
});

apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem("auth_token");
  if (token && config.headers) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default apiClient;
```

1.3 Role-Based Route Redirection

Upon authentication, query the user profile at `GET /api/v1/me/`. Inspect the `groups` array to determine the target portal destination:

```
interface UserProfile {
  id: number;
  username: string;
  email: string;
  groups: string[];
  is_superuser: boolean;
}

export function resolvePortalRoute(user: UserProfile): string {
  if (user.groups.includes("Students")) {
    return "/student/clearance";
  }

  const signatoryRoles = [
    "Librarians",
    "Cashiers",
    "Advisers",
    "Governors",
    "Deans",
    "Program Chairs",
  ];

  if (user.groups.some((role) => signatoryRoles.includes(role))) {
    return "/signatory/queue";
  }

  if (
    user.groups.includes("Clearance Management System Admins") ||
    user.is_superuser
  ) {
    return "/admin/dashboard";
  }

  return "/unauthorized";
}
```

---

2. Student Portal Integration

2.1 Fetch Active Clearance and Requirement Checklist

- **Screen:** Student Clearance Dashboard (`S-1`)
- **Endpoint:** `GET /api/v1/student-clearances/my-clearances/`
- **Description:** Retrieves the active clearance record for the logged-in student, including the status of each sequential requirement.
- **Response Payload (Array):**

```
[
  {
    "id": 1,
    "clearance_type_name": "Semestral Clearance",
    "academic_year": "2026-2027",
    "semester": "1st Semester",
    "overall_status": "in_progress",
    "is_enrolled": false,
    "items": [
      {
        "id": 1,
        "requirement_name": "Library Clearance",
        "authorizing_role_name": "Librarians",
        "order_sequence": 1,
        "status": "pending",
        "amount_due": "0.00",
        "remarks": null,
        "cleared_at": null
      },
      {
        "id": 2,
        "requirement_name": "Cashier Clearance",
        "authorizing_role_name": "Cashiers",
        "order_sequence": 2,
        "status": "deficiency",
        "amount_due": "150.00",
        "remarks": "Overdue book penalty: Database System Concepts",
        "proof_attachment": null,
        "proof_notes": null
      }
    ]
  }
]
```

2.2 Upload Settlement Proof

- **Screen / Dialog:** Settlement Proof Submission Dialog (`S-2`)
- **Endpoint:** `POST /api/v1/student-clearance-items/{id}/upload-proof/`
- **Request Format:** `multipart/form-data`
- **Media Handling:** The backend automatically uploads the binary file to institutional Cloudinary storage (`ubxjjws9`) and returns the permanent HTTPS CDN asset URL.
- **Form Parameters:**
  - `proof_attachment` (File, required): Image (JPEG/PNG) or PDF document.
  - `proof_notes` (String, optional): Transaction reference numbers or settlement details.
- **Client Implementation:**

```
async function submitSettlementProof(
  itemId: number,
  file: File,
  notes: string
) {
  const formData = new FormData();
  formData.append("proof_attachment", file);
  formData.append("proof_notes", notes);

  const response = await apiClient.post(
    `/student-clearance-items/${itemId}/upload-proof/`,
    formData,
    {
      headers: {
        "Content-Type": "multipart/form-data",
      },
    }
  );
  return response.data;
}
```

- **Resulting State:** Item `status` transitions to `"under_review"`.

2.3 Download Certificate of Clearance (PDF)

- **Screen:** Student Clearance Dashboard (`S-1`)
- **Endpoint:** `GET /api/v1/student-clearances/{id}/certificate/`
- **Condition:** Enabled exclusively when `overall_status === "completed"`.
- **Client Implementation:**

```
async function downloadClearanceCertificate(clearanceId: number) {
  const response = await apiClient.get(
    `/student-clearances/${clearanceId}/certificate/`,
    {
      responseType: "blob",
    }
  );

  const blob = new Blob([response.data], { type: "application/pdf" });
  const url = window.URL.createObjectURL(blob);
  const link = document.createElement("a");
  link.href = url;
  link.setAttribute("download", `Clearance_Certificate_${clearanceId}.pdf`);
  document.body.appendChild(link);
  link.click();
  link.remove();
  window.URL.revokeObjectURL(url);
}
```

2.4 Public QR Verification

- **Screen:** Public QR Verification Screen (`S-3`)
- **Endpoint:** `GET /api/v1/student-clearances/verify/{verification_token}/`
- **Authorization:** None (Public Endpoint).
- **Sample Production Response:**

```
{
  "student_name": "Froyd D. Carbajosa",
  "student_id": "202400633",
  "program": "Bachelor of Science in Information Technology",
  "clearance_type": "Semestral Clearance",
  "academic_year": "2026-2027",
  "semester": "1st Semester",
  "completion_date": "2026-11-06T10:15:00Z",
  "verification_token": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "is_authentic": true
}
```

---

3. Clearance Signatory Portal Integration

3.1 Signatory Workload Metrics

- **Screen:** Signatory Overview Dashboard (`SIG-1`)
- **Endpoint:** `GET /api/v1/student-clearance-items/signatory-dashboard/`
- **Response Payload:**

```
{
  "total_assigned": 36,
  "pending_count": 36,
  "deficiency_count": 0,
  "under_review_count": 0,
  "cleared_count": 0
}
```

3.2 Clearance Approval Queue

- **Screen:** Clearance Approval Queue Table (`SIG-2`)
- **Endpoint:** `GET /api/v1/student-clearance-items/`
- **Supported Query Parameters:**
  - `status`: Filter by status (`pending`, `deficiency`, `under_review`, `cleared`).
  - `search`: Filter by student full name or student ID number.
  - `page` and `page_size`: Pagination parameters.
- **Sample Response Item:**

```
{
  "id": 2,
  "student": {
    "student_id": "202400633",
    "full_name": "Froyd D. Carbajosa",
    "program_code": "BSIT",
    "year_level": 3,
```

```
    "section": "C"
  },
  "requirement_name": "Cashier Clearance",
  "status": "under_review",
  "amount_due": "150.00",
  "remarks": "Overdue book penalty: Database System Concepts",
  "proof_attachment": "https://res.cloudinary.com/ubxjjws9/image/upload/v1/receipt.jpg",
  "proof_notes": "Paid via GCash Ref 901234567890",
  "is_locked": false
}
```

3.3 Approve / Clear Requirement

- **Trigger:** Clicked on "Approve / Clear" in the queue table or settlement review dialog.
- **Endpoint:** `PATCH /api/v1/student-clearance-items/{id}/`
- **Request Payload:**

```
{
  "status": "cleared"
}
```

3.4 Flag Deficiency

- **Trigger:** Submitted via Flag Deficiency Dialog (`SIG-3`).
- **Endpoint:** `PATCH /api/v1/student-clearance-items/{id}/`
- **Request Payload:**

```
{
  "status": "deficiency",
  "amount_due": 150.0,
  "remarks": "Overdue book penalty: Database System Concepts"
}
```

Note: Dispatches an asynchronous email notification to the student automatically.

3.5 Program Chair Exclusive Endpoints

Program Chair Cohort Dashboard

- **Screen:** Program Chair Cohort Overview (`SIG-5`)
- **Endpoint:** `GET /api/v1/student-clearances/program-chair-dashboard/`
- **Permission:** Restricted to accounts holding `Roles.PROGRAM_CHAIR` or administrative roles.
- **Returns:** Program-wide clearance analytics and roster completion tracking for all students enrolled in the department.

Validate Student for Enrollment

- **Trigger:** Clicked on "Validate for Enrollment" button for a fully cleared student.
- **Endpoint:** `POST /api/v1/student-clearances/{id}/enroll/`
- **Request Payload:** Empty JSON object `{}`.
- **Result:** Sets `is_enrolled = true` on the target `StudentClearance` record and attributes the action to the authenticated Program Chair.

---

4. Administration Portal Integration

4.1 Campus-Wide Analytics Summary

- **Screen:** Executive Analytics Dashboard (`A-1`)
- **Endpoint:** `GET /api/v1/clearance-types/admin-summary/`
- **Returns:** Total enrolled students, completion percentages, outstanding liabilities total, and bottleneck office breakdown.

4.2 Issue Clearance Batch

- **Trigger:** Clicked via "Issue Clearance to Students" in Template Management (`A-2`).
- **Endpoint:** `POST /api/v1/clearance-types/{id}/issue/`
- **Request Payload:** Empty JSON object `{}`.
- **Result:** Initiates batch generation of student clearance instances and requirement items for all eligible students.

4.3 Bulk Student Roster Import

- **Screen / Modal:** Student Directory Import Dialog (`A-3`)
- **Upload Endpoint:** `POST /api/v1/admin/student-bulk-invitations/`
- **Request Format:** `multipart/form-data` with `file: students.xlsx`
- **Template Retrieval:** `GET /api/v1/admin/student-bulk-invitations/template/` (Returns downloadable `.xlsx` template).

4.4 Audit Logs and Report Exports

- **Screen:** Audit Logs Interface (`A-4`)
- **Data Log Query:** `GET /api/v1/audit-logs/?page=1&action=SIGN`
- **PDF Report Stream:** `GET /api/v1/audit-logs/report/pdf/`
- **Excel Report Stream:** `GET /api/v1/audit-logs/report/excel/`

---

5. Critical Frontend Implementation Rules

5.1 Enforcing Sequential Approval in the UI

The backend validates sequential order (`validate_sequential_clearance`). Attempting to sign Step 2 when Step 1 is incomplete returns an HTTP `400 Bad Request`.

- **Frontend Handling:** Check the `order_sequence` and statuses of prior items. If preceding items are not `cleared`, render the approval action button in a **disabled state** with an explanatory tooltip:

```
function canSignRequirement(
  currentItem: ClearanceItem,
  allItems: ClearanceItem[]
): boolean {
  const previousIncomplete = allItems.some(
    (item) =>
      item.order_sequence < currentItem.order_sequence &&
      item.status !== "cleared"
  );
  return !previousIncomplete;
}
```

5.2 Multipart Form Data for File Uploads

When submitting settlement proof documents, always instantiate a standard `FormData` object and omit manual `Content-Type` headers so the browser automatically sets the correct boundary string:

```
const formData = new FormData();
formData.append("proof_attachment", selectedFile);
if (notes) {
  formData.append("proof_notes", notes);
}
```

5.3 Cache Invalidation and Optimistic Updates

Upon successful execution of a sign-off (`PATCH`), deficiency flag (`PATCH`), or enrollment validation (`POST`), immediately trigger a query refetch for the respective view to ensure table counters and status badges reflect real-time backend state.

---

6. Seeded Production Test Accounts

The production database hosted on Aiven is pre-seeded with active test accounts for integration testing:

6.1 Student Test Accounts (BSIT 3rd Year, Set C)

All student passwords are: `password123`

Student ID	Login Username	Full Name	Program	Assigned Requirements
202400633	Student202400633	Froyd D. Carbajosa	BSIT 3-C	6 Sequential Items
202401033	Student202401033	Chieruf Jhann E. Pastedeo	BSIT 3-C	6 Sequential Items
202401164	Student202401164	Shane Angelo C. Garcia	BSIT 3-C	6 Sequential Items
202301626	Student202301626	Erick Cyle A. Embodo	BSIT 3-C	6 Sequential Items
202401177	Student202401177	Cyrus M. Suelto	BSIT 3-C	6 Sequential Items
202400928	Student202400928	Roosevelt O. Gulanes	BSIT 3-C	6 Sequential Items

6.2 Administrative and Staff Accounts

Default seeded password for all administrative accounts: `password123`

Username	Role	Scope
superadmin	Superuser / Platform Admin	Full Platform
dnsc_ic_admin	Institute Admin (`Institute Admins`)	Institute of Computing